Synthetic Pool Generator

Build a pool of random RNA sequences from a template of segments, with optional GC/AU/GU composition filtering, and write it out as FASTA.

Overview

The Synthetic Pool Generator builds random RNA libraries. You describe a sequence as a list of segments, some random (a region of a given length where every position is a uniform draw from A/C/G/U) and some fixed (a literal IUPAC motif), then ask for N copies. RSAS resolves the template once per sequence, optionally checks each result against composition targets, and writes the survivors to a FASTA file.

This is the tool you reach for when you want a designed-but-randomized starting set: a selection or screening library where part of the molecule is held constant and the rest is variable. A classic shape is a long random region followed by a fixed ribosome binding site and start codon, so you get a pool of candidate leaders that all share the functional handle but differ everywhere else.

The page lives in the left sidebar under **Synthetic Pool**.

When to use it

Reach for it when you need a large set of sequences that share structure but not identity:

- An in-silico SELEX or screening pool: a fixed motif (RBS, start codon, primer site, aptamer constant region) flanked or preceded by random variable regions.
- A randomized control or null set to fold through the analyzer and compare against your real sequences.
- A composition-controlled library: only keep sequences whose overall GC% (or AU%, or GU%) lands in a target band, useful when you want to hold base composition roughly constant across the pool.

If you just need one or two known sequences, type them into a FASTA file directly. This tool is for bulk randomized generation.

Step by step

1. **Build a template.** Add segments to the Sequence Template card. Each segment is either Random (you give a length) or Fixed (you give an IUPAC motif). Order them the way the sequence should

read, 5' to 3'. The live preview under the card shows the layout, for example `R(84) + GGAGG + R(8) + AUG = 100 nt`. You can also pick a built-in preset from the dropdown and click **Load Preset** to populate the template for you.

2. **Set size and seed.** In Pool Settings, set **Pool Size** (how many sequences to attempt) and, if you want reproducible output, a **Random Seed**. Leave the seed blank for a fresh random pool each run.
3. **Optionally filter by composition.** In Composition Targets, tick GC, AU, and/or GU, then enter a target percent and a tolerance for each you enable. A sequence is kept only if every enabled target is satisfied. Leave them all unchecked to keep every sequence.
4. **Choose an output file.** Click **Browse** in the Output card and pick where the FASTA goes (default name `synthetic_pool.fasta` in your Downloads folder), or type the path.
5. **Generate.** Click **Generate**. Progress streams into the Progress box, and when it finishes you get a summary dialog with how many sequences were written and how many were filtered out.

Generation runs on a background thread, so the dialog stays responsive while a large pool is being written.

Options in detail

Segments

A template is an ordered list of segments. There are two types:

- **Random**: you give a positive integer **Length**. RSAS fills that many positions, each an independent uniform pick from A, C, G, U (`random_region`).
- **Fixed**: you give a **Motif** in IUPAC code. Plain bases (`A`, `C`, `G`, `U`) are written through literally; `T` is read as `U`. Degenerate codes expand to a random matching base per position (for example `R` becomes A or G, `N` becomes any of A/C/G/U, `S` becomes G or C). Motifs are uppercased and validated when you generate; an invalid IUPAC character stops the run with an error naming the segment.

The segment type dropdown switches the field label between **Length:** and **Motif:** and clears the field, so switching types does not leave a stale value behind.

The 10-segment cap

A template can hold at most **10 segments**. Adding an eleventh shows a "Maximum 10 segments" warning and is ignored. Use **+ Add Segment** to append a row and **Remove Last** to drop the bottom one.

Drag to reorder

Each segment row has a drag handle (the dotted grip on the left). Grab it and drag a row up or down to change segment order without deleting anything; the target row highlights as you drag, and the **Seg N:**

labels renumber once you drop. Reordering rebuilds the rows in place, so the field values move with them.

Presets

The preset dropdown offers **Custom** plus any built-in presets. Selecting **Custom** and clicking **Load Preset** clears the template. The shipped preset is **RBS + AUG**, which loads a 100 nt layout: a random region of 84, the fixed motif `GGAGG`, a random region of 8, and the fixed start codon `AUG` (`R(84) + GGAGG + R(8) + AUG = 100 nt`).

Pool size

Default 1000000 (one million). This is the number of sequences RSAS *attempts*. Without composition filtering, attempts equal written. With filtering on, some attempts can be rejected, so the number written may be lower. Pool size must be a positive integer.

Random seed

Optional. Leave it blank and the pool is seeded from system randomness (different every run). Enter an integer and the pool is fully reproducible: the same seed, template, and targets produce the same FASTA byte for byte. The seed must be an integer if provided.

Composition targets and tolerances

Three independent targets, each with a checkbox, a **Target** percent, and a **± tolerance** percent (tolerance defaults to `5`):

- **GC%**: fraction of G plus C.
- **AU%**: fraction of A plus U.
- **GU%**: fraction of G plus U.

A target only applies when its checkbox is ticked, which also enables its two entry fields. For each enabled target a sequence passes when `|actual - target| <= tolerance`, with everything measured in percent. All enabled targets must pass for the sequence to be kept. Targets must be between 0 and 100; tolerance must be non-negative.

Important: composition is measured over the **whole** sequence, random and fixed segments combined. The dialog shows an orange warning to that effect right above the target rows. See Limitations below for why this matters.

What you get

A FASTA file at the path you chose. Each written sequence is two lines, a header and the sequence. The header packs in the composition and template that produced it:

```
>synth_pool_seq_1|len=100|gc=0.480|au=0.520|gu=0.510|segments=R84_GGAGG_R8_AUG
GCUA...AUG
```

The fields are the running written-sequence index, the length, the measured GC/AU/GU fractions (each to three decimals), and a compact tag of the template (random segments as `R<length>`, fixed segments as the motif, joined with underscores).

A result summary. `generate_pool` returns a dictionary with four keys:

- `total`: the pool size you requested (number of attempts).
- `written`: how many sequences were actually written.
- `failed`: how many attempts were rejected by composition filters.
- `file`: the output path.

The dialog turns this into a "Generation Complete" message showing sequences written and, if any were rejected, the filtered count. If every attempt was filtered out (`written == 0` and `failed > 0`), you get a warning instead, telling you the composition constraints are too strict and suggesting you relax the target, widen the tolerance, or shorten the fixed segments.

How it works

The core is `generate_pool(n, segments, output_file, *, targets, seed, progress_callback)` in `RnaThermofinder/utils/synthetic_pool_generator.py`.

1. It creates one `random.Random(seed)` generator and builds the segment tag once.
2. It opens the output file and loops `n` times. For each iteration it calls `generate_single_sequence`, which resolves the template into a concrete sequence: random segments become uniform A/C/G/U strings, and fixed segments have each IUPAC character resolved to a random matching base per sequence (so `GGAGG` is always `GGAGG`, but a degenerate motif like `RYN` varies from sequence to sequence).
3. If composition targets are set, `generate_single_sequence` uses **rejection sampling**: it rebuilds the whole sequence and re-checks until it passes, up to `max_tries` (1000) attempts. If no attempt within that budget satisfies the targets, it returns `None`, and `generate_pool` counts that as a `failed` sequence and moves on.
4. Each kept sequence is written with its computed header, and progress is reported roughly every 1% of the pool (and on the final sequence).

So each sequence is an independent draw, degenerate codes are resolved fresh per sequence, and composition filtering is per-sequence rejection sampling, not a global rebalancing of the pool.

Worked example

Generate a small, reproducible pool from the shipped preset.

1. Pick **RBS + AUG** from the preset dropdown and click **Load Preset**. The template fills with four segments and the preview reads `R(84) + GGAGG + R(8) + AUG = 100 nt`: 84 random bases, the fixed `GGAGG`, 8 random bases, then the fixed start codon `AUG`.
2. Set **Pool Size** to `10`.
3. Set **Random Seed** to a fixed integer, say `42`, so you can reproduce the result.
4. Leave the composition targets unchecked (no filtering).
5. Browse to an output file and click **Generate**.

You get a FASTA with 10 sequences, each 100 nt long, each sharing `GGAGG` at positions 85 to 89 and `AUG` at the end, with the 84 and 8 nt windows randomized. The summary reports `written = 10`, `failed = 0`. Because the seed is fixed, running it again with the same template and seed reproduces the same 10 sequences exactly.

Tips

- **Lock in reproducibility with a seed.** Any time you want to share, regenerate, or debug a pool, set an integer seed. Same seed plus same template plus same targets gives identical output.
- **Read the preview before generating.** The `R(...) + ... = N nt` line is the fastest check that your segment order and lengths are what you intended, especially after a few reorders.
- **Use degenerate motifs for constrained variability.** A fixed segment does not have to be literal. `N`, `R`, `Y`, `S`, `W`, `K`, `M`, and the three-way codes let you randomize within a position while keeping the segment "fixed" in the template, which is handy for partially constrained constant regions.
- **Start small when filtering.** If you are using composition targets, run a small pool first to see the filtered count before committing to a million attempts.
- **The header carries the metadata.** Composition and template tag travel with every sequence, so downstream you can read GC/AU/GU off the FASTA header without recomputing.

Limitations and gotchas

- **Composition is over the whole sequence, not just the random regions.** GC/AU/GU are measured on the full resolved sequence, including the fixed motifs. If your fixed segments are GC-rich or GU-heavy, they pull the whole-sequence composition with them, and a target that ignores them will be hard or impossible to hit. With the RBS + AUG preset, for instance, `GGAGG` and `AUG` are baked into every sequence, so any GC% target has to account for those bases. The dialog warns about this in orange above the target rows.

- **Tight targets can stall or never finish.** Filtering is rejection sampling with a per-sequence budget of 1000 tries. A narrow tolerance, an extreme target, or a target the fixed motifs make unreachable means many or all sequences exhaust their tries and count as `failed`. In the worst case every attempt is rejected and nothing is written, and a million attempts of 1000 tries each is a lot of wasted work. Keep tolerances generous and targets realistic for your length and fixed content.
- **Conflicting targets compound.** Enabling GC, AU, and GU at once with tight tolerances narrows the acceptable region sharply, since all enabled targets must pass together. The more you enable and the tighter you set them, the more attempts get rejected.
- **No de-duplication.** Sequences are independent draws and are not checked for uniqueness. For short random regions or very large pools you can get repeats.
- **At most 10 segments.** Templates beyond ten segments are not supported.

Troubleshooting

- **"Invalid Segment" on generate.** A random segment needs a positive integer length, and a fixed segment needs a non-empty motif using valid IUPAC characters (A, C, G, U/T, R, Y, S, W, K, M, B, D, H, V, N). The error names the offending segment number.
- **"No Segments."** The template is empty. Add at least one segment, or load a preset.
- **"Invalid Pool Size."** Pool size must be a positive integer.
- **"Invalid Seed."** The seed must be an integer, or left blank for a random pool.
- **"Invalid Target" / "Invalid Tolerance."** For each enabled composition target, the target must be a number from 0 to 100 and the tolerance must be a non-negative number.
- **"No Output File."** Choose an output path with Browse, or type one, before generating.
- **"No Sequences Written" warning.** Every attempted sequence was filtered out by the composition constraints. Relax the target percent, increase the tolerance, or shorten the fixed segments so the whole-sequence composition can reach your target.
- **Fewer sequences than the pool size.** Expected when filtering is on: the difference is the `failed` count, the attempts that could not meet the targets within their try budget. The summary reports both the written and filtered counts.